Lab: Exploring Stack-Based Bytecode with Compiler Explorer

Duration: 2 hours **Tool:** Compiler Explorer (Godbolt)

Objectives

By the end of this lab, you should be able to:

- recognize an operand-stack-based virtual machine;
- read simple bytecode and reason about its operand stack;
- relate source-level constructs to bytecode instructions;
- compare bytecode generated by different compilers and languages;
- investigate unfamiliar instructions using official VM documentation;
- compare several real-world stack-based virtual machines;
- understand that different source languages can compile to the same VM while producing different bytecode.

---

1. Background

Real virtual machines are sophisticated, but several widely used VMs are also based on an **operand stack**.

In this lab, you will investigate some of such systems:

Source language	Target	VM
Java	JVM bytecode	Java Virtual Machine
Kotlin	JVM bytecode	Java Virtual Machine
C#	CIL	.NET Common Intermediate Language
C / Rust	WebAssembly	WebAssembly VM

You will use Compiler Explorer to see what actual compilers produce.

> **Important:** We are interested in stack-based execution. Do not select LLVM IR, native assembly, or other register-based intermediate representations for this lab.

---

2. Getting Started with Compiler Explorer

Open:

https://godbolt.org/

For each experiment:

1. Select the source language on the left.
2. Select an appropriate compiler.
3. Select the target/output corresponding to the VM being studied.
4. Keep the source programs as small as possible.
5. Read the generated bytecode from top to bottom.
6. Look up unfamiliar instructions in the **official VM documentation**.

When possible, enable compiler options that make the generated code easier to read.

Exercise 1 — Java → JVM: Arithmetic

Time: ~20 minutes

Start with Java targeting JVM bytecode (JDK 25.0.1).

Use:

```
class Main {
    static int func(int x, int y) {
        return (x + 2) * y;
    }
}
```

1.1 Inspect the bytecode

Find the generated JVM bytecode for `func`.

Identify:

- how `x` is accessed;
- how `y` is accessed;
- how the constant `2` is pushed;
- which instruction performs the addition;
- which instruction performs the multiplication;
- which instruction returns the result.

You should be able to identify instructions corresponding to operations such as:

```
load
constant
add
multiply
return
```

Do not worry if the exact instruction names are unfamiliar at first.

1.2 Trace the operand stack

Assume:

```
x = 3
y = 5
```

The expression is:

```
(3 + 2) * 5
```

Draw the operand stack after every relevant instruction.

For example:

```
initial:       [...]
load x:        [..., 3]
push 2:        [..., 3, 2]
...
```

Complete the trace until the function returns.

1.3 Compare with the abstract machine

Recall our abstract instructions:

```
PUSH 2
ADD
MUL
```

Which JVM instructions play approximately the same role?

Construct a table:

Abstract machine	JVM
`PUSH 2`	?
`ADD`	?
`MUL`	?
return	?

1.4 Investigate constants

You will probably notice that the JVM does not have just one instruction for pushing an integer constant.

Investigate the JVM instructions used for small integer constants.

Why might a real VM provide several different instructions for loading constants instead of having only:

```
PUSH k
```

?

1.5 Documentation

Consult the **official JVM specification** and verify the stack behavior of at least two instructions you identified.

For each instruction, record:

```
Instruction:
Stack before:
Stack after:
Purpose:
```

Exercise 2 — Java → JVM: Control Flow and Special Instructions

Time: ~25 minutes

Now consider:

```
class Main {
    static int abs(int x) {
        if (x < 0)
            return -x;
        else
            return x;
    }
}
```

2.1 Identify the control-flow structure

Find the JVM bytecode corresponding to:

```
if (x < 0)
    ...
else
    ...
```

Identify:

- the comparison;
- the conditional branch;
- the two branches;
- the return instructions;
- any unconditional jumps.

Draw a simplified control-flow graph.

For example:

```
            [condition]
             /       \
            /         \
       negative    non-negative
          |            |
          v            v
       return       return
```

2.2 Trace both paths

Trace the operand stack for:

```
x = -5
```

and then for:

```
x = 5
```

At each branch instruction, answer:

> What is on top of the operand stack when the branch is evaluated?

2.3 Stack consistency at control-flow joins

Consider a bytecode program containing a merge point:

```
      branch
      /    \
     /      \
path A   path B
    \      /
     \    /
      merge
```

Why is it important that the operand stack has a valid and compatible shape when control flow reaches the merge point?

Think about what would happen if one path reached the merge with:

```
[..., value]
```

while another reached it with:

```
[...]
```

Write a short explanation.

2.4 Bytecode archaeology: `new`, `dup`, and method calls

Now compile:

```
class Main {
    static String hello() {
        return new String("hello");
    }
}
```

You will probably encounter instructions that are less obvious than:

```
iload
iadd
imul
ireturn
```

For example, you may encounter instructions related to:

```
new
dup
invokespecial
areturn
```

Pick **one unfamiliar instruction**.

Using the official JVM specification, determine:

1. What does the instruction do?
2. What does it consume from the operand stack?
3. What does it produce on the operand stack?
4. Why is the instruction needed here?
5. Why can't this operation be represented simply by `PUSH`?

Record your answer as:

```
Instruction:
Purpose:

Stack before:
Stack after:

Why is it needed?
```

Exercise 3 — Java vs Kotlin: Same JVM, Different Compilation

Time: ~20 minutes

This exercise explores an important compiler-design question:

> **If two different languages target the same virtual machine, do they generate the same bytecode?**

Use both Java and Kotlin.

3.1 Same computation, two languages

Java

```
class Main {
    static int f(int x, int y) {
        return (x + 2) * y;
    }
}
```

Kotlin

```
fun f(x: Int, y: Int): Int {
    return (x + 2) * y
}
```

Compile both to JVM bytecode using Compiler Explorer.

3.2 Compare the generated bytecode

Create a comparison table:

Question	Java	Kotlin
How is `x` loaded?		
How is `y` loaded?		
How is `2` loaded?		
Addition		
Multiplication		
Return		

Then answer:

> Are the generated bytecodes identical?

If not:

> Which parts are different?

3.3 Compare evaluation order

For both implementations, determine the operand-stack evolution for:

```
x = 3
y = 5
```

Do the Java and Kotlin compilers use the same evaluation order?

For example, does the stack evolve conceptually as:

```
[..., x, 2][]
[..., x + 2][]
[..., x + 2, y][]
[..., (x + 2) * y]
```

?

3.4 A more interesting example

Now compare:

Java

```
class Main {
    static int max(int x, int y) {
        return x > y ? x : y;
    }
}
```

Kotlin

```
fun max(x: Int, y: Int): Int =
    if (x > y) x else y
```

Inspect both bytecode outputs.

Answer:

1. Are the source programs structurally identical?
2. Does the generated JVM bytecode preserve the difference?
3. How is the conditional represented?
4. Which instructions perform the comparison?
5. How are the two possible values selected?

3.5 Compiler or language?

Suppose Java and Kotlin produce different bytecode.

Which of these statements is the best explanation?

```
A. The JVM requires different bytecode for Java and Kotlin.

B. Java and Kotlin have different JVMs.

C. Different compilers are free to implement the same source-level
   semantics using different valid JVM instruction sequences.

D. JVM bytecode directly represents the source language.
```

Choose the best answer and explain it.

Optional extension — A less familiar JVM language

If you finish early, try **Clojure**. Clojure is another JVM language that compiles a little bit differently.

Implement the equivalent of:

```
(x + 2) * y
```

and inspect the generated JVM bytecode.

Ask:

> Is the generated code still as simple as the Java/Kotlin version?

If it is substantially more complicated, investigate why.

Consider whether the extra instructions are caused by:

- the language runtime;
- function representation;
- dynamic features;
- boxing;
- method calls;
- compiler strategy.

Do not try to understand every instruction. Pick **one interesting difference** and investigate it.

Exercise 4 — C# → CIL

Time: ~20 minutes

Now leave the JVM and investigate another stack-based intermediate language: **.NET CIL**.

Use C#:

```
class Main
{
    static int f(int x, int y)
    {
        if (x < y)
            return x + 1;
        else
            return y * 2;
    }
}
```

Compile to CIL/IL in Compiler Explorer.

4.1 Identify the stack operations

Find instructions corresponding to:

- accessing `x`;
- accessing `y`;
- pushing `1`;
- pushing `2`;
- comparison;
- addition;
- multiplication;
- conditional branching;

- returning.

Create a table:

Source operation	CIL instruction
access `x`	
access `y`	
push `1`	
`x < y`	
`x + 1`	
`y * 2`	
return	

4.2 Trace one path

Choose:

```
x = 3
y = 5
```

Trace the operand stack through the `x < y` branch and the `x + 1` computation.

4.3 Compare JVM and CIL

Compare this program with the equivalent JVM program.

Identify three similarities and three differences.

Similarities

```
1.
2.
3.
```

Differences

```
1.
2.
3.
```

Then answer:

> Why can both JVM bytecode and CIL be described as stack-based intermediate languages even though their instruction sets are different?

4.4 Investigate one CIL instruction

Pick one instruction that you did not immediately understand.

Use the **official .NET documentation** to determine:

```
Instruction:
Purpose:

Stack before:
Stack after:

Why is this instruction useful?
```

Exercise 5 — C → WebAssembly

Time: ~20 minutes

Now investigate WebAssembly.

WebAssembly uses a stack-based execution model, although it also has concepts such

as locals.

Use:

```
int f(int x, int y)
{
    return (x + 2) * y;
}
```

Compile it to WebAssembly in Compiler Explorer.

5.1 Identify the instructions

Find the WebAssembly instructions corresponding to:

- accessing `x`;
- accessing `y`;
- pushing `2`;
- addition;
- multiplication;
- return.

You may encounter instructions such as:

```
local.get
i32.const
i32.add
i32.mul
```

Identify their stack behavior.

5.2 Trace the stack

For:

```
x = 3
y = 5
```

trace the WebAssembly operand stack.

For example:

```
local.get x
```

should conceptually produce:

```
[..., 3]
```

Continue until the result is produced.

5.3 Compare three VMs

You have now seen:

```
JVM
CIL
WebAssembly
```

For the computation:

```
(x + 2) * y
```

compare the three instruction sequences.

Operation	JVM	CIL	WebAssembly
Load `x`			
Push `2`			
Add			
Load `y`			
Multiply			
Return			

5.4 Control flow in WebAssembly

Now investigate how WebAssembly represents loops.

Consider:

```
int sum_to(int n)
{
    int sum = 0;

    while (n > 0) {
        sum += n;
        n--;
    }

    return sum;
}
```

Compile it to WebAssembly.

Find instructions related to:

```
block
loop
br
br_if
```

You do **not** need to understand every generated instruction.

Instead, answer:

1. Where is the loop represented?
2. What instruction causes control to go back to the beginning?
3. What instruction conditionally exits or continues the loop?
4. How does this differ from the JVM conditional-jump representation?

Exercise 6 — Bytecode Archaeology

Time: ~10 minutes

Return to **one of the VMs** you have investigated.

Choose a bytecode sequence from one of the previous exercises.

Find an instruction that you still do not fully understand.

Examples might include:

```
JVM:
    invokedynamic
    invokespecial
    checkcast
    getstatic

CIL:
    box
    callvirt
    conv.*
    ldloc

WebAssembly:
    block
    br_if
    local.tee
    i32.wrap_i64
```

Choose **one**.

Use the official documentation for that VM to answer:

1. What is its purpose?

```
...
```

2. What does it consume?

```
Stack before:
...
```

3. What does it produce?

```
Stack after:
...
```

4. Why did the compiler generate it?

```
...
```

5. Could our tiny abstract machine represent the same operation?

If not:

> What would we need to add to our abstract machine?

Final Discussion — What Have We Learned?

~10 minutes

Discuss the following questions.

Question 1 — Why stack machines?

Why might a VM designer choose an operand-stack architecture?

Consider:

- instruction encoding;
- simplicity;
- portability;
- compiler implementation;
- interpretation;
- verification;

- execution.

Question 2 — Same idea, different instruction sets

The JVM, CIL, and WebAssembly all use stack-based execution, but they have different instructions.

Why isn't there a single universal stack-machine instruction set?

Question 3 — Same VM, different languages

Java and Kotlin both target the JVM.

Why don't they necessarily generate identical bytecode?

What does this tell us about the role of a compiler?

Question 4 — Source information

Consider the following source program:

```
return (x + 2) * y;
```

After compilation to JVM bytecode, what information about the original source program is still visible?

What information has disappeared?

For example:

```
Still visible:

- arithmetic operations
- control flow
- types
- ...

No longer directly visible:

- original formatting
- variable names
- source-level expression structure
- ...
```

Question 5 — Back to our abstract machine

Compare our tiny machine:

```
PUSH
POP
DUP
SWAP

ADD
SUB
MUL
MOD

EQ
LT

LABEL
JUMP
JUMP_IF_ZERO
```

with a real VM such as the JVM.

Why does the real VM need many more instructions?

What kinds of operations are missing from our machine?

Deliverable

Submit a short report containing:

Part 1 — Stack traces

One complete operand-stack trace for:

```
(x + 2) * y
```

on the JVM.

Part 2 — Control flow

One annotated bytecode sequence showing how an `if` statement is implemented.

Part 3 — Different languages

A comparison of the Java and Kotlin bytecode for the same computation.

Include at least **one concrete difference** and explain why it occurs.

Part 4 — Different VMs

A comparison of:

```
JVM
CIL
WebAssembly
```

Include at least two similarities and two differences.

Part 5 — Bytecode archaeology

Document one unfamiliar instruction using the official documentation:

```
Instruction:
VM:
Purpose:
Stack before:
Stack after:
Why the compiler generated it:
```

Suggested Timing

Activity	Time
Introduction + Compiler Explorer setup	10 min
Exercise 1 — Java/JVM arithmetic	20 min
Exercise 2 — Java/JVM control flow	25 min
Exercise 3 — Java vs Kotlin	20 min
Exercise 4 — C#/CIL	20 min
Exercise 5 — WebAssembly	15 min
Exercise 6 + final discussion	10 min
Total	**120 min**

Recommended strategy

Do not try to understand every generated instruction.

The goal is to develop the ability to:

> **read a stack-based bytecode sequence, reconstruct its stack behavior, and investigate unfamiliar instructions using the VM specification.**

When you encounter a complicated sequence, first reduce it to the essential operations:

```
load value
push constant
perform operation
branch
call
return
```

Then investigate the details.